

Boom Barrier Gate Controller

ESP32 integration guide for the Goodwish Smart Parking system

Online card checks, offline fallback, and log syncing

Document version 1.1 · 17 September 2026

Audience: firmware developer building the gate controller

System: Goodwish Engineering parking management (Django API, one site per mall)

New in 1.1: the exit gate, online and offline (section 7); the card list now has `exit_only_cards` ; offline logic and sample firmware updated (two hex characters per byte, card list read in small pieces); corrected examples, time zones, trailing-slash behaviour and troubleshooting.

1. What you are building

A member taps an RFID card on the reader at the barrier. Your ESP32 decides whether the barrier opens.

1. Read the card UID from the reader.
2. Ask the parking server: is this card allowed, for entry or for exit?
3. If the server answers `open_barrier: true` , pulse the relay and open the barrier.
4. If the internet is down, decide locally from a card list you downloaded earlier, record what you did, and upload those records when the connection returns.

```
Card tap
|
v
[ESP32] --online--> POST /gate-access/ --> server answers open_barrier true/false
|                                     (the server also records the tap)
|
|--offline--> check the downloaded card list --> decide locally
               store the event in a queue
               when back online: POST /sync-offline-logs/ then GET /active-cards/
```

There are two controllers per lane: one at the entry barrier and one at the exit barrier. They run the same firmware with one setting changed. The exit gate has its own rules, explained in section 7.

The server owns the member database, the validity dates and the anti-passback state (whether a member's vehicle is currently inside). The barrier hardware, the reader and the local fallback are yours.

Three endpoints, that is all

Endpoint	Method	Use it for	How often
/gate-access/	POST	One card tap, online decision	Every tap
/active-cards/	GET	Download the card list for offline use	Every 5-10 minutes, and at boot
/sync-offline-logs/	POST	Upload taps you handled offline	As soon as the connection returns

2. Before you start

2.1 Base address

Every mall (tenant) has its own address and its own gate key. All paths below are relative to:

```
https://<mall>.wheelstrulyours.com.np/api/membership
```

Example for the Central Mall site:

```
https://centralmall.wheelstrulyours.com.np/api/membership/gate-access/
```

Always keep the trailing slash. Without it the request fails. Today the server answers with an HTTP 500 error page (after a planned server setting change it would become a 301 redirect, which the ESP32 will not follow with the POST body either). Because the sample firmware treats any answer other than 200, 400 or 403 as "server down", a missing slash makes the gate quietly run offline all the time, even with perfect internet.

2.2 Gate key

Every request must carry the mall's gate key in a header named `X-Gate-Key`. Ask the parking administrator for it: in the admin website it is under *Member* → *Gate Logs* → *Gate Key*.

```
X-Gate-Key: 0123456789abcdef0123456789abcdef
```

- The key is a 32-character hex string, and it is the same for every controller at that mall.
- The key does not change during normal operation, so you program it once.
- If a key ever has to be replaced, for example after a controller is stolen, the administrator asks the software developer to issue a new one, and every controller must then be updated.

Treat it like a password. Anyone holding this key can list all active cards and open gates. Do not commit it to Git and do not paste it into chat groups. Prefer storing it in NVS (section 9.4) over hard-coding it in a sketch you share.

2.3 No login, no tokens

These three endpoints do not use the staff login system. The gate key alone identifies your controller. An `Authorization` header is harmless; it is ignored.

2.4 HTTPS

The API is HTTPS only, on port 443, behind Cloudflare. On the ESP32 use `WiFiClientSecure`. Either pin a root certificate with `setCACert()`, or, if certificate handling is a problem during development, use `setInsecure()`, remembering that it skips certificate checking.

2.5 What you need from the mall side

Item	Where it comes from
Mall address (<mall>.wheelstrulyyours.com.np)	Parking administrator
Gate key	Admin website, Gate Key window
One test member card	Parking staff register a test member and assign a card
Wi-Fi credentials at the gate	Mall IT

3. API 1 — Card tap (online decision)

POST /api/membership/gate-access/

3.1 Request

```
POST /api/membership/gate-access/ HTTP/1.1
Host: centralmall.wheelstrulyyours.com.np
Content-Type: application/json
X-Gate-Key: <gate key>

{
  "card_uid": "76A0FFA9",
  "gate_type": "ENTRY"
}
```

Field	Type	Required	Notes
card_uid	string	Yes	The UID exactly as your reader prints it. See section 6 for formats.
gate_type	string	No	"ENTRY" or "EXIT" . If you leave it out, the tap counts as ENTRY . No other value is accepted.

There is one controller per direction: an entry barrier always sends ENTRY , an exit barrier always sends EXIT . Any other value is refused with HTTP 400.

3.2 Successful answer (barrier opens)

```
HTTP/1.1 200 OK

{
  "status": "SUCCESS",
  "access_granted": true,
  "open_barrier": true,
  "action": "ENTRY",
  "message": "Welcome, Rajesh Shrestha!",
  "data": {
    "member_name": "Rajesh Shrestha",
    "phone_number": "9841234567",
    "membership_type": "monthly outhouse",
    "vehicles": ["BA 2 KHA 9922"],
    "is_inside": true,
    "valid_until": "2083-06-28",
    "days_remaining": 28,
    "pass_status": "ACTIVE",
    "gate_type": "ENTRY",
    "timestamp": "2026-09-16T04:27:44.812345+00:00"
  }
}
```

Rule for the firmware: open the barrier when, and only when, open_barrier is true . Never decide from the status text or from the HTTP code alone.

`valid_until` is a Nepali (Bikram Sambat) date, meant for a display or a voice prompt. Do not do date maths on it; use `days_remaining` if you want to warn a member that their pass ends soon. `pass_status` is `ACTIVE` for a normal tap; at the exit it can also be `EXPIRED` or `NOT_YET_ACTIVE` (section 7). Every date-time the API sends (`timestamp` , `validity` , `last_activity`) is in UTC, marked `+00:00` ; Nepal time is 5 hours 45 minutes ahead.

3.3 Refusals (barrier stays closed)

These all arrive as **HTTP 200** with `open_barrier: false` . They are normal answers, not errors.

status	Meaning	Extra data	Suggested display
<code>INVALID_CARD</code>	Card is not registered at this mall	—	"Card not registered"
<code>EXPIRED</code>	Membership has ended. At the exit, only when the server does not have this car inside (section 7)	<code>member_name</code> , <code>phone_number</code> , <code>valid_until</code> , <code>days_remaining: 0</code>	"Pass expired, renew at counter"
<code>NOT_YET_ACTIVE</code>	Membership starts on a later date. At the exit, same condition as above	<code>member_name</code>	"Pass not active yet"
<code>ANTI_PASSBACK_ERROR</code>	Entry tap while the server already has this member inside	<code>member_name</code> , <code>last_activity</code>	"Vehicle already inside"

```
{
  "status": "ANTI_PASSBACK_ERROR",
  "access_granted": false,
  "open_barrier": false,
  "message": "Vehicle is already recorded as INSIDE the parking lot. Entry denied.",
  "data": { "member_name": "Rajesh Shrestha", "last_activity": "2026-09-16T03:55:02+00:00" }
}
```

Anti-passback applies to entry only. At the exit, a member with a valid pass is always let out, and a member whose pass ended while the car was parked is let out too. Both mark the member as outside. Section 7 has the full exit rules.

3.4 Errors

HTTP	When	Body	What the firmware should do
400	card_uid missing, or gate_type not ENTRY/EXIT	{"status": "INVALID_REQUEST", ...}	Fix the request; this is a firmware bug
403	Gate key missing or wrong	{"detail": "Missing or invalid gate key."}	Stop and alert. Do not fall back to offline mode; the key is wrong
500	Trailing slash missing (section 2.1)	HTML error page	Looks like a server fault but is a URL bug. If every tap gets this while the internet works, check the slash first
404	Wrong path	HTML	Fix the URL
5xx	Server problem, or a software update in progress	Any	Treat as offline: decide locally and queue the log
timeout	No internet, Wi-Fi lost, DNS failure	—	Treat as offline (section 8)

The server restarts for a short time whenever a software update is released, which can happen several times a week. Your offline path covers that automatically, which is why every controller needs one.

3.5 Timing

A normal answer takes well under a second. Measured on 17 September 2026: about 0.4 s when each tap opens a new HTTPS connection, and about 0.12 s when the connection is kept open and reused. Use a request timeout of about 3 to 5 seconds, so a driver never waits long at a closed barrier; after the timeout, use the offline path.

4. API 2 — Download active cards (for offline use)

GET /api/membership/active-cards/

4.1 Request

```
GET /api/membership/active-cards/ HTTP/1.1
Host: centralmall.wheelstrulyyours.com.np
X-Gate-Key: <gate key>
```

4.2 Answer

```
HTTP/1.1 200 OK

{
  "status": "SUCCESS",
  "total_active_cards": 128,
  "timestamp": "2026-09-16T04:30:00.123456+00:00",
  "cards": [
    {
      "card_uid": "76A0FFA9",
      "card_variants": ["0010551209", "10551209", "1990262697", "2852102262", "76A0FFA9",
        "76a0ffa9", "A0FFA9", "A9FFA076", "a9ffa076"],
      "member_name": "Rajesh Shrestha",
      "validity": "2026-10-14T18:14:59.999999+00:00",
      "is_inside": false,
      "vehicles": ["BA 2 KHA 9922"]
    }
  ],
  "exit_only_cards": [
    {
      "card_uid": "04A1B2C3",
      "card_variants": ["..."],
      "member_name": "Sita Gurung",
      "validity": "2026-09-15T18:14:59.999999+00:00",
      "is_inside": true,
      "vehicles": ["BA 3 PA 4411"]
    }
  ]
}
```

Field	Meaning
<code>card_uid</code>	The UID as stored by the staff
<code>card_variants</code>	The same card written in other formats: hex in both cases, decimal, reversed byte order, 3-byte forms. Compare your reader's UID against this list, so its format does not matter.
<code>validity</code>	End of the last valid day (midnight Nepal time), written in UTC. Compare your clock against this for offline decisions.
<code>is_inside</code>	Whether the server currently considers this member inside. Useful if you enforce anti-passback offline.
<code>vehicles</code>	Plate numbers, for display only
<code>exit_only_cards</code>	Same fields. Members whose car the server has inside although the pass has ended or not started yet. Only the exit controller uses this list , so those cars can still leave while offline.

`cards` contains only members who have a card, whose pass has not expired, and whose start date has arrived; `total_active_cards` counts this list. An expired card moves to `exit_only_cards` while its car is inside, and disappears once the car has left. Deleted or unassigned cards disappear from both lists.

4.3 How to use it

- Fetch at boot, then every 5 to 10 minutes.
- Fetch again right after uploading offline logs, so your copy of `is_inside` is current.
- Keep the previous copy if a fetch fails. Never wipe your stored list because one request failed.
- The simple safe way: write the download to a new file, for example `/cards.new`. Only when it is complete, rename it to `/cards.dat`; with LittleFS the rename replaces the old file in one step, even if the power fails at that moment. If the download breaks, delete `/cards.new`; the old file was never touched. The ESP32 therefore holds one list at a time and its storage does not fill up.
- Entry controller: store `cards` only. Exit controller: store `cards` and `exit_only_cards`, remembering which list each card came from.
- Expect a few hundred cards per mall. The download is large: about 300 bytes of JSON per card, so 300 cards is roughly 90 KB. That is too much to hold in memory at once on a normal ESP32, so read it one card at a time (section 9.3).
- In flash, about 200 bytes per card is enough for the UID, its variants (around nine short strings), the validity and the flags.

This list is a fallback, not the normal path. While the internet is up, always ask `/gate-access/`, because only the server knows about a pass renewed or cancelled a minute ago.

5. API 3 — Upload offline taps

POST /api/membership/sync-offline-logs/

5.1 Request

```
POST /api/membership/sync-offline-logs/ HTTP/1.1
Content-Type: application/json
X-Gate-Key: <gate key>

{
  "logs": [
    { "card_uid": "76A0FFA9", "gate_type": "ENTRY", "status": "GRANTED", "seconds_ago": 540, "reason":
      "offline tap" },
    { "card_uid": "76A0FFA9", "gate_type": "EXIT", "status": "GRANTED", "seconds_ago": 120 },
    { "card_uid": "11223344", "gate_type": "ENTRY", "status": "DENIED_INVALID_CARD", "seconds_ago": 60 }
  ]
}
```

Field	Required	Notes
card_uid	Yes	Entries without it are skipped
gate_type	Yes	ENTRY or EXIT ; anything else is stored as ENTRY
status	No	Defaults to GRANTED . Use the values in section 12.2 to record what you actually did.
seconds_ago	Recommended	How many seconds before <i>this upload</i> the tap happened. The best choice for a board with no battery-backed clock.
timestamp	Alternative	ISO 8601 text, or Unix seconds or milliseconds. Use it only if your board keeps real time.
reason	No	Free text, shown to staff in the logs

Use seconds_ago . Store `millis()` at the moment of the tap, and at upload time send `(millis() - tapMillis) / 1000` . The server converts it into a real time, so an ESP32 without an RTC still produces correct logs.

5.2 Answer

```
HTTP/1.1 200 OK

{
  "status": "SUCCESS",
  "synced_records": 3,
  "errors": ["Card 11223344 was not registered."],
  "message": "Successfully synced 3 offline gate log(s)."
```

- `errors` lists taps by unknown cards. Those taps are still recorded for the staff, so this is information, not a failure.
- An empty list, or a missing `logs` field, gives HTTP 400.

- Clear your queue only after you receive HTTP 200. On a timeout, keep the queue and retry.

5.3 What the server does with them

- It sorts all entries by time before applying them, so entry and exit taps land in the right order even when you send them out of order.
- A `GRANTED` entry marks the member inside and a `GRANTED` exit marks them outside, but only when the tap is newer than that member's last recorded gate activity. Older taps are stored without changing the current state.
- Every entry appears in the admin website under *Member* → *Gate Logs* (the page titled Member Gate Access Logs).

6. Card numbers (UID formats)

Readers print the same card in different ways: hex `76A0FFA9` , reversed hex `A9FFA076` , decimal `1990262697` , zero-padded decimal `0012345678` , or with separators `76:A0:FF:A9` .

You do not need to know your reader's byte order. The staff's desk reader types decimal numbers, for example `2851937782` , which is the card bytes `A9 FD 1D F6` . That card is recognised whether your reader gives `A9FD1DF6` or `F61DFDA9` , online and offline.

Write every byte as two characters. A byte of value 5 must become `05` , not `5` . The common Arduino line `Serial.print(uid[i], HEX)` drops that zero, so a card with bytes `A9 05 1D F6` comes out as `A951DF6` instead of `A9051DF6` . Neither the server nor the stored list will recognise it. Cards without a small byte still work, which makes this bug look random.

```
// Card bytes to text: always two characters per byte, upper case, no separators
String uidToHex(const byte* uid, byte size) {
    String out;
    char two[3];
    for (byte i = 0; i < size; i++) {
        snprintf(two, sizeof(two), "%02X", uid[i]);    // 0x05 -> "05"
        out += two;
    }
    return out;                                        // 4-byte card: 8 characters, 7-byte card: 14
}
```

6.1 Online

Send the UID exactly as your reader gives it. The server generates the equivalent forms itself (hex and decimal, both byte orders, 3, 4 and 7 byte cards, with or without separators) and matches any of them. Upper or lower case does not matter.

6.2 Offline

Do the same comparison locally with the `card_variants` list from `/active-cards/` :

1. Take the UID from the reader.
2. Remove spaces, colons, dashes and underscores, and convert to upper case.
3. Compare it against `card_uid` and every entry of `card_variants` , each normalised the same way.

Keep one format inside your firmware, ideally upper-case hex with no separators, and normalise at the single point where you read the card. That keeps the comparison and the logs consistent.

6.3 Member cards versus visitor cards

The same MIFARE Classic 1K cards are used for visitors at the counter, where the staff app writes ticket data into the card's memory. Member cards are recognised by their UID only, so the gate never needs to read or write card memory. Read the UID and nothing else.

7. The exit gate, online and offline

The exit controller runs the same firmware with `GATE_TYPE = "EXIT"`. What changes is the goal: an exit barrier should almost never keep a car inside.

The rule: let the car out if its card is valid, or if the car came in on that card and the pass ended while it was parked. Keep the barrier closed only for unknown cards, and for ended passes whose car did not come in on the card.

7.1 Online (the normal case)

Send the tap with `"gate_type": "EXIT"` and open when `open_barrier` is `true`, exactly as at the entry. The server applies the rule:

Card	Server has the car inside?	Answer	Barrier	Suggested display
Valid pass	Either	<code>SUCCESS</code> , <code>pass_status: ACTIVE</code>	Opens	"Goodbye"
Pass ended	Yes	<code>SUCCESS</code> , <code>pass_status: EXPIRED</code>	Opens	"Pass expired, please renew"
Pass not started yet	Yes	<code>SUCCESS</code> , <code>pass_status: NOT_YET_ACTIVE</code>	Opens	"Pass not active yet"
Pass ended	No	<code>EXPIRED</code>	Closed	"Pass expired, renew at counter"
Pass not started yet	No	<code>NOT_YET_ACTIVE</code>	Closed	"Pass not active yet"
Unknown card	—	<code>INVALID_CARD</code>	Closed	"Card not registered"

```
{
  "status": "SUCCESS",
  "access_granted": true,
  "open_barrier": true,
  "action": "EXIT",
  "message": "Goodbye, Sita Gurung! Your pass expired on 2083-05-30. Please renew.",
  "data": {
    "member_name": "Sita Gurung",
    "is_inside": false,
    "valid_until": "2083-05-30",
    "days_remaining": 0,
    "pass_status": "EXPIRED",
    "gate_type": "EXIT",
    ...
  }
}
```

- Every granted exit marks the member as outside and appears in the gate logs. An exit is never refused for anti-passback.
- A long screen can show `message` as it is. A small screen can pick a short text from `pass_status`.

- After an exit on an ended pass, the card no longer opens the entry: it answers `EXPIRED` until the member renews.

Why "came in on that card": without it, someone whose pass has ended could come in as a visitor and leave through the member exit without paying the visitor fee. The server knows the car is inside because the entry tap marked it so.

7.2 Offline

The exit controller decides from its stored lists. It does not check the date and does not check `is_inside`, because its offline copy may be out of date and a wrong refusal traps a car.

Card found in	Barrier	Queue as	Suggested display
<code>cards</code> , validity still in the future	Opens	<code>GRANTED</code>	"Goodbye"
<code>cards</code> , validity has passed since the download	Opens	<code>GRANTED</code>	"Pass expired, please renew"
<code>exit_only_cards</code>	Opens	<code>GRANTED</code>	"Pass expired, please renew"
Neither list	Closed	<code>DENIED_INVALID_CARD</code>	"Card not accepted, please see staff"

When the queue is uploaded, each `GRANTED` exit marks that member as outside on the server (section 5.3).

7.3 The entry gate, for comparison

- **Online:** the server decides. Expired, not-yet-active and already-inside cards are refused.
- **Offline:** use `cards` only and ignore `exit_only_cards`. Refuse a card whose validity has passed by your clock. Do not refuse because of the stored `is_inside` (section 8.5).

7.4 When staff must step in

A car can still be held at the exit in rare cases: a damaged card, a card that is in neither stored list during a long outage, or an ended pass whose car came in another way. Staff open the barrier by hand. Do not add a firmware shortcut that opens for any card.

8. Offline mode

Two situations must not stop the gate: the mall's internet drops, and the server restarts during a software update. Both look the same to you, a failed request.

8.1 Decision flow for one tap

```
read UID
|
|-- online? try POST /gate-access/ (timeout 3-5 s)
|   |
|   |-- HTTP 200 -> open barrier if open_barrier == true, show message, done
|   |-- HTTP 403 -> key problem: refuse, light the error LED, alert staff
|   |-- HTTP 400 -> firmware bug: refuse, log locally
|   |-- 5xx / timeout / no Wi-Fi -> fall through to offline
|
|-- offline, ENTRY controller (uses "cards" only)
|   |
|   |-- card found?          no -> refuse, queue DENIED_INVALID_CARD
|   |-- validity still in the future? no -> refuse, queue DENIED_EXPIRED
|   |-- otherwise           -> open barrier, queue GRANTED
|   | (the stored is_inside is not checked, see 8.5)
|
|-- offline, EXIT controller (uses "cards" and "exit_only_cards")
|   |
|   |-- card found in either list? no -> refuse, queue DENIED_INVALID_CARD
|   |-- otherwise             -> open barrier, queue GRANTED
|   | (validity and is_inside are not checked at the exit, see 7.2)
```

8.2 Queue

- Store each offline tap with the UID, direction, the status you applied and the `millis()` value at that moment.
- Keep the queue in flash (LittleFS or an SD card) so it survives a power cut.
- Keep it in its own file, for example `/queue.dat`, separate from the card list. Replacing the card list never touches the queue.
- Cap it, for example at 5,000 taps. At about 40 bytes per tap that is only 200 KB, and it covers several days offline at a busy gate. If it ever fills, drop the oldest and raise a warning; an outage that long needs staff attention anyway.

8.3 Coming back online

1. Upload the queue with `POST /sync-offline-logs/`, in batches of up to 50 taps, oldest first. Small batches keep memory use low.
2. Remove a batch from the queue only on HTTP 200.
3. When the queue is empty, call `GET /active-cards/` to refresh the list and the `is_inside` flags, so the refreshed flags already include your offline taps.

Order is safe. The server sorts the taps inside each batch by time, and a tap only changes a member's inside/outside state if it is newer than the last one the server knows. An old tap that arrives late, from either controller, is stored in the logs without undoing a newer one.

Offline taps are never lost by a list refresh. The taps and the card list are separate files, and the list is refreshed only after the queue has been uploaded and the server has answered HTTP 200. If the upload fails, the taps stay in the queue and the refresh waits for the next try.

8.4 Clock

The ESP32 loses the time when it restarts. Two options:

- **Simplest:** keep `millis()` per tap and send `seconds_ago`. No clock needed.
- **With NTP:** sync at boot and send a real `timestamp`. Needed anyway if you want the correct local date for offline validity checks.

For the offline validity check without NTP, remember the `timestamp` from the last successful `/active-cards/` call and add the time elapsed since. That is accurate enough to compare against a validity date. Both values are in UTC, so compare them directly without converting to Nepal time.

8.5 Anti-passback offline: leave it off

Offline, the entry controller should not refuse a card just because its stored `is_inside` says the car is inside. That copy goes out of date: if the member left through the exit controller while the internet was down, the entry controller never heard about it and would lock the member out. Open for any card in `cards` with a valid date, and let the server correct its state when the logs arrive. Use strict mode only if the mall asks for it. The exit controller never checks `is_inside`.

9. Reference firmware (ESP32, Arduino)

Sketch skeleton with the parts that talk to the server. Libraries: `WiFi`, `WiFiClientSecure`, `HTTPClient`, `ArduinoJson` (v7), `Preferences`.

9.1 Configuration and HTTP helper

```
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <HTTPClient.h>
#include <ArduinoJson.h>

static const char* API_BASE = "https://centralmall.wheelstrulyyours.com.np/api/membership";
static const char* GATE_TYPE = "ENTRY";           // "EXIT" on the exit controller
static const int RELAY_PIN = 26;
String gateKey;                                   // loaded from NVS, see 9.4

WiFiClientSecure tls;

// Returns the HTTP status code, or a negative number for a transport failure.
int apiCall(const char* method, const String& path, const String& body, String& response) {
    tls.setInsecure();                             // or tls.setCACert(ROOT_CA)
    HTTPClient http;
    http.setConnectTimeout(5000);
    http.setTimeout(5000);
    if (!http.begin(tls, String(API_BASE) + path)) return -1;
    http.addHeader("X-Gate-Key", gateKey);
    int code;
    if (strcmp(method, "POST") == 0) {
        http.addHeader("Content-Type", "application/json");
        code = http.POST(body);
    } else {
        code = http.GET();
    }
    if (code > 0) response = http.getString();
    http.end();
    return code;
}

void pulseBarrier() {
    digitalWrite(RELAY_PIN, HIGH);
    delay(500);                                     // match your barrier's input
    digitalWrite(RELAY_PIN, LOW);
}
```

9.2 One card tap

```
// true = handled online (barrier already opened or refused)
// false = server unreachable, caller should use the offline path
bool handleTapOnline(const String& uid) {
    JsonDocument req;
    req["card_uid"] = uid;
    req["gate_type"] = GATE_TYPE;
    String body, response;
    serializeJson(req, body);

    int code = apiCall("POST", "/gate-access/", body, response);

    if (code == 403) { showError("Gate key rejected"); return true; } // do not open
    if (code == 400) { showError("Bad request"); return true; }
    if (code != 200) return false; // 5xx or transport failure

    JsonDocument doc;
    if (deserializeJson(doc, response)) return false;

    bool open = doc["open_barrier"] | false;
    const char* message = doc["message"] | "";
    showMessage(message);
    if (open) pulseBarrier();
    return true;
}
```

9.3 Card list and log upload

The card list can be 100 KB or more, so it is read straight from the connection one card at a time, never loaded into one `String`. The server always sends the fields in this order: `status`, `total_active_cards`, `timestamp`, `cards`, `exit_only_cards`.

```

int waitPeek(Stream& s) {
    // next byte, without taking it
    unsigned long start = millis();
    while (!s.available() && millis() - start < 5000) delay(1);
    return s.peek();
}

bool readCardArray(Stream& body, const char* start, bool exitOnly, JsonDocument& filter) {
    if (!body.find(start)) return false;
    if (waitPeek(body) == ']') { body.read(); return true; } // empty list
    do {
        JsonDocument card;
        if (deserializeJson(card, body, DeserializationOption::Filter(filter))) return false;
        storeCard(card.as<JsonObject>(), exitOnly); // copy the fields and the exitOnly flag
    } while (body.findUntil(",", "[]"));
    return true;
}

bool refreshActiveCards() {
    tls.setInsecure(); // or tls.setCACert(ROOT_CA)
    HTTPClient http;
    http.useHTTP10(true); // plain reply body, no chunks to decode
    http.setConnectTimeout(5000);
    http.setTimeout(10000);
    if (!http.begin(tls, String(API_BASE) + "/active-cards/")) return false;
    http.addHeader("X-Gate-Key", gateKey);
    if (http.GET() != 200) { http.end(); return false; }

    Stream& body = http.getStream();
    JsonDocument filter; // keep only what the gate needs
    filter["card_uid"] = true;
    filter["card_variants"] = true;
    filter["validity"] = true;
    filter["is_inside"] = true;

    String listTime;
    bool ok = body.find("\"timestamp\":");
    if (ok) listTime = body.readStringUntil(' '); // UTC time of this list (section 8.4)

    beginNewCardList(); // open /cards.new; /cards.dat stays in use
    ok = ok && readCardArray(body, "\"cards\":[", false, filter);
    if (ok && strcmp(GATE_TYPE, "EXIT") == 0) { // the entry controller ignores this list
        ok = readCardArray(body, "\"exit_only_cards\":[", true, filter);
    }
    http.end();

    if (!ok) { discardNewCardList(); return false; } // delete /cards.new, keep /cards.dat
    commitNewCardList(); // LittleFS.rename("/cards.new", "/cards.dat")
    lastListTime = listTime;
    lastListMillis = millis();
    return true;
}

// Sends up to 50 taps, oldest first. Call it again while queueCount() > 0.
bool uploadOfflineQueue() {
    if (queueCount() == 0) return true;

    int batch = min(queueCount(), 50);
    JsonDocument req;
    JsonArray logs = req["logs"].to<JsonArray>();
    uint32_t now = millis();

```

```

for (int i = 0; i < batch; i++) {
    OfflineLog e = queueAt(i);
    JsonObject log = logs.add<JsonObject>();
    log["card_uid"] = e.uid;
    log["gate_type"] = e.gateType;
    log["status"] = e.status; // GRANTED, DENIED_EXPIRED, ...
    log["seconds_ago"] = (now - e.tapMillis) / 1000;
    log["reason"] = "offline tap";
}

String body, response;
serializeJson(req, body);
if (apiCall("POST", "/sync-offline-logs/", body, response) != 200) return false;

removeOldest(batch); // only after HTTP 200
return true;
}

```

9.4 Storing the gate key

```

#include <Preferences.h>
Preferences prefs;

void loadGateKey() {
    prefs.begin("gate", true);
    gateKey = prefs.getString("key", "");
    prefs.end();
}

void saveGateKey(const String& key) { // from a serial command or a setup page
    prefs.begin("gate", false);
    prefs.putString("key", key);
    prefs.end();
}

```

9.5 One card tap, offline

```
void handleTapOffline(const String& uid) {
    StoredCard* card = findCard(uid);           // compares with card_uid and card_variants
    bool isExit = strcmp(GATE_TYPE, "EXIT") == 0;

    if (card == nullptr) {
        showMessage(isExit ? "Card not accepted, please see staff" : "Card not registered");
        queueTap(uid, "DENIED_INVALID_CARD");
        return;
    }

    bool passEnded = card->exitOnly || validityPassed(card);    // UTC compare, section 8.4

    if (isExit) {
        // exit: never trap a car (section 7.2)
        showMessage(passEnded ? "Pass expired, please renew" : "Goodbye");
        pulseBarrier();
        queueTap(uid, "GRANTED");
        return;
    }

    if (passEnded) {
        // entry: the pass must still be valid
        showMessage("Pass expired, renew at counter");
        queueTap(uid, "DENIED_EXPIRED");
        return;
    }
    showMessage("Welcome");
    pulseBarrier();
    queueTap(uid, "GRANTED");
}
```

9.6 Main loop

```
// setup(): connect Wi-Fi, loadGateKey(), load the stored card list and queue from flash,
// then uploadOfflineQueue() and refreshActiveCards() once.

void loop() {
    if (WiFi.status() == WL_CONNECTED) {
        // every 10 minutes, or every 30 seconds while taps are waiting to be uploaded
        uint32_t every = queueCount() > 0 ? 30UL * 1000UL : 10UL * 60UL * 1000UL;
        if (millis() - lastSync > every) {
            lastSync = millis();
            while (queueCount() > 0 && uploadOfflineQueue()) {}    // batch by batch, stops on a failure
            if (queueCount() == 0) refreshActiveCards();           // upload first, then refresh
        }
    }

    String uid;
    if (readCardUid(uid)) {
        // your reader driver, built with uidToHex
        (section 6)
        uid = normaliseUid(uid);
        // upper case, no separators
        if (!WiFi.status() == WL_CONNECTED && handleTapOnline(uid)) {
            handleTapOffline(uid);
            // section 9.5
        }
    }
}
```

10. Error handling and retries

Situation	Barrier	Queue the tap?	Retry
HTTP 200, <code>open_barrier: true</code>	Open	No	—
HTTP 200, <code>open_barrier: false</code>	Closed	No, the server logged it	—
Timeout or no Wi-Fi	Offline decision	Yes	Upload on next sync
HTTP 5xx	Offline decision	Yes	Next sync; a deploy usually ends within a minute
HTTP 403	Closed	No	Do not retry in a loop; alert staff, the key is wrong
HTTP 400	Closed	No	Firmware bug, fix the request
HTTP 500 on every tap, internet fine	Offline decision	Yes	Check the trailing slash (section 2.1)
Malformed JSON	Offline decision	Yes	Next sync

- Back off between failed syncs, for example 30 s, 1 min, 5 min, then every 10 minutes, so a long outage does not hammer the link.
- There is no rate limit on these three endpoints, but one request per tap is plenty.
- Log locally (serial or a small file) how many taps were handled offline; it makes site visits much easier.

11. Testing

11.1 From a laptop first

Check the key and the address before flashing anything. Replace the key and the host.

```
curl -X POST https://centralmall.wheelstrulyours.com.np/api/membership/gate-access/ \
-H "Content-Type: application/json" \
-H "X-Gate-Key: PASTE_KEY_HERE" \
-d '{"card_uid":"76A0FFA9","gate_type":"ENTRY"}'

curl https://centralmall.wheelstrulyours.com.np/api/membership/active-cards/ \
-H "X-Gate-Key: PASTE_KEY_HERE"
```

A wrong key gives `{"detail":"Missing or invalid gate key."}` with HTTP 403.

11.2 Checklist on the bench

#	Test	Expected
1	Tap a valid member card, entry controller	<code>open_barrier: true</code> , barrier opens
2	Tap the same card again without exiting	<code>ANTI_PASSBACK_ERROR</code> , barrier stays closed
3	Tap it on the exit controller	Granted, member becomes outside; entry works again afterwards
4	Tap an unknown card	<code>INVALID_CARD</code> , barrier stays closed
5	Enter with a test card, ask staff to set its end date to yesterday (Edit Details), then tap at the exit	<code>SUCCESS</code> with <code>pass_status: EXPIRED</code> , barrier opens, display asks to renew
6	Tap that card at the entry	<code>EXPIRED</code> , barrier stays closed
7	Tap it at the exit again (the car is now outside)	<code>EXPIRED</code> , barrier stays closed
8	Unplug the router, tap a valid card at the entry	Barrier opens from the stored list, tap is queued
9	Router still unplugged, tap the same card at the exit	Barrier opens, tap is queued
10	Before unplugging: enter, expire the pass as in test 5, let the exit controller refresh its list; then unplug and tap at the exit	Card is in <code>exit_only_cards</code> , barrier opens, tap is queued
11	Plug the router back in	Queue uploads, taps appear in Member → Gate Logs, members show as outside
12	Power cut with a full queue, then power on	Queue survives and uploads
13	Wrong gate key	HTTP 403, barrier stays closed, staff alerted
14	Remove the trailing slash from the address	HTTP 500 on every tap: confirms you recognise this mistake
15	Tap 5 or more different cards and look at the UID on the serial monitor	Every 4-byte card gives exactly 8 characters (7-byte cards: 14)
16	Let the controller download the full card list, then check free memory (<code>ESP.getFreeHeap()</code>)	Download succeeds and free memory returns to about the same level afterwards

Ask the staff to confirm every test in the admin website under *Member → Gate Logs*, where each tap shows with its time, card, direction and result.

11.3 Troubleshooting

Symptom	Likely cause
HTTP 403 on every request	Key copied with a space, or from a different mall
Gate always works offline although the internet is fine; every tap gets HTTP 500	Missing trailing slash at the end of the path (section 2.1)
Some cards work, others always get <code>INVALID_CARD</code>	A byte printed without its leading zero, for example <code>5</code> instead of <code>05</code> (section 6)
Card list download fails or the board restarts during it	The whole list was loaded into memory at once; read it card by card (section 9.3)
TLS connect fails	Certificate not set; use <code>setCACert()</code> , or <code>setInsecure()</code> while developing
Valid card reported as <code>INVALID_CARD</code>	The card was never assigned to a member, or it was assigned at a different mall
Entry refused with "Vehicle already inside" although the car is outside	The server never recorded the exit (the car left while the barrier was held open by hand, or an offline queue was lost). Staff fix it in the admin website: <i>Members list → : menu on that member → Reset Gate State</i> . The option only appears while the member is marked inside.
Exit refused with <code>EXPIRED</code>	The pass has ended and the server does not have this car inside, so it did not come in on this card. Staff check and open by hand
Times wrong in the logs	<code>seconds_ago</code> computed from the wrong reference, or a stale <code>millis()</code> after a restart

12. Appendix

12.1 Statuses from `/gate-access/`

Status	HTTP	open_barrier
SUCCESS	200	true
INVALID_CARD	200	false
EXPIRED	200	false
NOT_YET_ACTIVE	200	false
ANTI_PASSBACK_ERROR	200	false
INVALID_REQUEST	400	false

SUCCESS at the exit can come with `pass_status` EXPIRED or NOT_YET_ACTIVE : the car is let out and the member should renew.

12.2 Status values for offline logs

Value	Use when you
GRANTED	Opened the barrier offline, at the entry or the exit, including an exit on an ended pass
DENIED_INVALID_CARD	Did not find the card in your stored list
DENIED_EXPIRED	Found it, but its validity had passed
DENIED_NOT_ACTIVE	Found it, but its pass had not started
DENIED_ANTI_PASSBACK	Refused a second entry for a member you had inside

12.3 Words used here

Word	Meaning
Member	A person with a monthly parking pass and an RFID card
Mall / tenant	One parking site. Each has its own web address, members and gate key
Anti-passback	Blocking a second entry until the vehicle has exited, so one card cannot admit several cars
Card UID	The unique number in the RFID chip. The gate identifies members only by this
BS date	Bikram Sambat, the Nepali calendar, used for display
Exit-only card	A card in <code>exit_only_cards</code> : its car is inside but the pass has ended or not started, so it may leave but not enter
Admin website	Where staff manage members, cards, renewals, gate logs and the gate key
POS app	The Android app staff use at the counter for visitors. It has no member features

12.4 What to ask the software team for

- The mall address and the gate key (and a separate test mall, if one is available).
- A test member with a card you can keep.
- Access to the admin website's gate logs page, or a staff member who can read it out during testing.
- Notice before a software update, if you are testing that day.

Summary of the contract between us: you send the card UID and the direction with the gate key; the server answers `open_barrier` . When the server cannot be reached, you decide from the downloaded card lists and send the taps afterwards. At the exit, a car that came in on a card is let out even if the pass has ended. Nothing else is required from the controller.